WEEK 3 · LESSON 17 OF 24 · HANDOUT

Security for AI Systems

Study time: **about 55 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=nErkn4Na7ZE — open the lesson online to watch inside the course.

🎓 **Go deeper:** EuroPython 2023: Poisoned pickles make you ill — https://www.youtube.com/watch?v=c2c9lqKb39I

🎓 **Go deeper:** LiveOverflow: Attacking LLM — Prompt Injection — https://www.youtube.com/watch?v=Sv5OLj2nVAQ

Your goal in this lesson: Identify the main attacks on ML systems, see poisoning, gaming and model copying in action, and protect a model API with authentication, rate limits and safe loading.

The threat model

Attack	What the attacker does	Defence
Data poisoning	Inserts false training data or labels	Control and validate data sources, track changes
Evasion / gaming	Changes inputs cheaply to get approved	Use verified, hard-to-fake features; monitor
Model extraction	Queries the API many times to copy the model	Authentication, rate limits, coarse outputs
Membership inference	Learns whether a person was in the training data	Less overfitting, differential privacy
Unsafe model files	Sends a model file that runs code when loaded	Only load trusted files; prefer safe formats
Prompt injection (LLM apps)	Hides instructions in input text	Course 6, Lesson 23
Classic attacks	Stolen keys, open ports, old libraries	Secrets management, updates, least privilege

```
import numpy as np
import pandas as pd

def make_applicants(n=3000, seed=7, drift=0.0, concept=0.0):
    # Made-up SACCO loan applications. drift > 0: incomes, loans and late payments change.
    # concept > 0: the link to repayment changes (salaried jobs protect less, late payments
hurt more).
    rng = np.random.default_rng(seed)
    gender = rng.choice(["female", "male"], n)
    area = rng.choice(["urban", "rural"], n, p=[0.6, 0.4])
    rural = area == "rural"
    income = (rng.lognormal(12.4, 0.5, n) * np.where(rural, 0.7, 1.0) * (1 - drift *
0.25)).round(-3)
    loan = (rng.uniform(50_000, 600_000, n) * (1 + drift)).round(-3)
    years = rng.integers(0, 13, n)
    late = rng.poisson(0.7 + drift, n)
    salaried = (rng.random(n) < np.where(rural, 0.3, 0.6)).astype(int)
    momo = np.clip(rng.normal(np.where(rural, 18, 45), 10), 0, None).round().astype(int)
    score = 3.0 - 1.6 * (loan / income) + 0.12 * years - 0.7 * late + 0.6 * salaried + 0.01 *
momo
    score = score - concept * (1.5 * salaried + 0.6 * late - 0.04 * (momo - 30))
    repaid = (rng.random(n) < 1 / (1 + np.exp(-score))).astype(int)
    officer = score - 0.9 * rural - 0.6 * (gender == "female") + rng.normal(0, 0.6, n)
    return pd.DataFrame({
        "applicant_id": np.arange(1, n + 1), "age": rng.integers(20, 63, n), "gender": gender,
"area": area,
        "income_rwf": income, "loan_rwf": loan, "years_member": years, "late_payments": late,
        "salaried": salaried, "momo_tx_month": momo,
        "approved_before": (officer > 1.2).astype(int), "repaid": repaid})

applicants = make_applicants()
print(applicants.shape, "| repaid:", round(applicants["repaid"].mean(), 3))
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["income_rwf", "loan_rwf", "years_member", "late_payments", "salaried",
"momo_tx_month"]
TARGET = "repaid"
train, test = train_test_split(applicants, test_size=0.25, random_state=0,
stratify=applicants[TARGET])
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(train[FEATURES], train[TARGET])
proba = model.predict_proba(test[FEATURES])[:, 1]
print(f"accuracy {accuracy_score(test[TARGET], proba >= 0.5):.3f} | AUC
{roc_auc_score(test[TARGET], proba):.3f}")
```

Poisoning: an insider flips labels

Someone with access to the training data marks many rural repayers as "defaulted":

```
rng = np.random.default_rng(3)
poisoned = train.copy()
targets = poisoned.index[(poisoned["area"] == "rural") & (poisoned[TARGET] == 1)]
flip = rng.choice(targets, size=int(0.6 * len(targets)), replace=False)
poisoned.loc[flip, TARGET] = 0

bad = make_pipeline(StandardScaler(),
LogisticRegression(max_iter=1000)).fit(poisoned[FEATURES], poisoned[TARGET])
for name, m in [("clean model", model), ("poisoned model", bad)]:
    approved = m.predict_proba(test[FEATURES])[:, 1] >= 0.5
    rural = (test["area"] == "rural").values
    print(f"{name:15s} AUC {roc_auc_score(test[TARGET], m.predict_proba(test[FEATURES])[:,
1]):.3f} | "
          f"rural approval {approved[rural].mean():.1%} | urban approval
{approved[~rural].mean():.1%}")
```

The overall AUC drops only a little — it would still pass a 0.80 quality gate — but rural approvals collapse to about a third of what they were. Defences: restrict who can change training data, keep a change log, compare label rates per group between data versions, and run the fairness audit (Lessons 13–14) before every release.

Gaming: cheap features can be faked

Mobile-money activity is easy to inflate: someone can make many tiny transfers before applying.

```
applicant = {"income_rwf": 200_000, "loan_rwf": 420_000, "years_member": 1, "late_payments": 1,
"salaried": 0, "momo_tx_month": 15}
for tx in [15, 60, 120]:
    p = model.predict_proba(pd.DataFrame([dict(applicant, momo_tx_month=tx)])[FEATURES])[0, 1]
    print(f"{tx:3d} transactions/month -> repay probability {p:.0%}")
```

Defences: prefer features that are verified or costly to fake (repayment history, verified salary), cap how much one feature can add, and watch for sudden jumps in a feature before applications.

Model extraction: copying through the API

```
from sklearn.tree import DecisionTreeClassifier

queries = make_applicants(n=2000, seed=77)[FEATURES]                # attacker invents
applicants
answers = model.predict(queries)                                    # ... and records the
API's decisions
copy = DecisionTreeClassifier(max_depth=6, random_state=0).fit(queries, answers)
fresh = make_applicants(n=2000, seed=78)[FEATURES]
print(f"the copied model agrees with ours on {(copy.predict(fresh) ==
model.predict(fresh)).mean():.1%} of new applicants")
```

With 2,000 free queries, an attacker gets a close copy — which they can use to find exactly how to game it. Defences: require an **API key**, **rate-limit** each key, return only the decision (not the exact probability) to outside users, and watch for unusual query patterns.

A protected API

```
import secrets
import time
from collections import defaultdict, deque
from fastapi import Depends, FastAPI, Header, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel, Field

API_KEYS = {"branch-kigali": "kgl-" + secrets.token_hex(8)}          # in real life: from a
secret store
calls = defaultdict(deque)
LIMIT, WINDOW = 5, 60

def check_key(x_api_key: str = Header(default="")):
    for client_name, key in API_KEYS.items():
        if secrets.compare_digest(x_api_key, key):                  # constant-time comparison
            now = time.time(); q = calls[client_name]
            while q and now - q[0] > WINDOW:
                q.popleft()
            if len(q) >= LIMIT:
                raise HTTPException(429, "Too many requests")
            q.append(now)
            return client_name
    raise HTTPException(401, "Missing or wrong API key")

class Applicant(BaseModel):
    income_rwf: float = Field(..., ge=10_000, le=20_000_000)
    loan_rwf: float = Field(..., ge=10_000, le=5_000_000)
    years_member: int = Field(..., ge=0, le=60)
    late_payments: int = Field(..., ge=0, le=50)
    salaried: int = Field(..., ge=0, le=1)
    momo_tx_month: int = Field(..., ge=0, le=1000)

app = FastAPI()

@app.post("/decide")
def decide(a: Applicant, client_name: str = Depends(check_key)):
    p = model.predict_proba(pd.DataFrame([a.model_dump()])[FEATURES])[0, 1]
    return {"decision": "approve" if p >= 0.5 else "refer to officer"}     # no raw probability
outside

api = TestClient(app)
body = dict(applicant, momo_tx_month=15)
print("no key:   ", api.post("/decide", json=body).status_code)
key = {"X-API-Key": API_KEYS["branch-kigali"]}
print("with key: ", [api.post("/decide", json=body, headers=key).status_code for _ in
range(7)])
```

No key → **401**. With a key, the sixth call within a minute → **429 Too Many Requests**.

Never load untrusted model files

`joblib` and `pickle` files can run code when loaded. This harmless demo only prints a message — a real attacker could do anything:

```
import pickle

class Surprise:
    def __reduce__(self):
        return (print, ("!!! code ran while loading this file — it could have been anything
!!!",))

data = pickle.dumps(Surprise())
pickle.loads(data)
```

Only load model files you created or that come from a trusted, checked source; verify a checksum before loading; and prefer safe formats (for example ONNX or safetensors) when sharing models.

Key terms

- **Threat model** — a list of who might attack, how, and the defences.
- **Data poisoning** — corrupting training data to change the model.
- **Evasion / gaming** — changing inputs to get a desired output.
- **Model extraction** — copying a model by querying it.
- **Rate limiting** — capping requests per client per time window.
- **Constant-time comparison** — comparing secrets without leaking timing information.

Remember this: AI systems face poisoning, gaming, copying and unsafe files on top of classic attacks — defend with controlled data, robust features, authentication, rate limits, coarse outputs and trusted artifacts.

Name: _____ Date: _____

Activity

Task 1

Poison only 20% of rural repayer labels. Would your Week 2 quality gate notice? Would the fairness audit?

Task 2

Make the copied model with 200 queries instead of 2,000. How much does agreement drop?

Task 3

Add a second API key for another branch and check each branch has its own limit.

Task 4

Write a threat model table for a USSD loan application service.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is data poisoning?
   - A. Food safety
   - B. A slow database
   - C. Deleting logs
   - D. Corrupting training data or labels to change the model
2. Why was the poisoning hard to notice from AUC alone?
   - A. AUC is not computed
   - B. The model crashed
   - C. AUC went to zero
   - D. Overall AUC dropped only a little while one group was harmed a lot
3. Why is mobile-money activity a risky feature?
   - A. It is cheap to inflate, so people can game the model
   - B. It is always missing
   - C. It has no values
   - D. It is illegal
4. How can attackers copy a model through an API?
   - A. By reading the logo
   - B. By guessing the password
   - C. By sending many queries and training on the answers
   - D. They cannot
5. Which status code means too many requests?
   - A. 200
   - B. 401
   - C. 422
   - D. 429
6. Why never load untrusted pickle or joblib files?
   - A. They are slow
   - B. They can run code when loaded
   - C. They are large
   - D. They lose accuracy

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.